المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⴰⵍⵉⵏ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

ENSA - BM

École Nationale des Sciences Appliquées

RAPPORT DE TRAVAUX PRATIQUES

Docker - Application Multi-Conteneurs

Étudiant	Fouad BANAN
Module	Génie Logiciel
Filière	Transformation Digitale Industrielle
Encadrant	Pr. BE.ELBAGHAZAOUI
Établissement	ENSA-BM
Année universitaire	2025 / 2026

1. Introduction

Ce rapport présente le compte-rendu du travail pratique réalisé dans le cadre du module Génie Logiciel, portant sur la conteneurisation d'une application multi-services à l'aide de Docker. L'objectif principal était de dockeriser une application web composée d'un serveur Nginx pour le frontend et d'une base de données PostgreSQL pour le backend, tout en utilisant Docker Compose pour orchestrer l'ensemble des services.

Ce TP met en pratique les concepts fondamentaux de Docker, notamment la création d'images personnalisées via un Dockerfile, la gestion de volumes persistants, la configuration de réseaux inter-conteneurs et l'utilisation de fichiers d'environnement sécurisés.

Objectifs du TP

- Maîtriser la création d'un Dockerfile personnalisé
- Configurer et orchestrer plusieurs services via docker-compose.yml
- Mettre en place une base de données PostgreSQL conteneurisée
- Administrer la base de données via pgAdmin
- Gérer la persistance des données avec des volumes Docker
- Sécuriser les configurations via des fichiers .env

2. Environnement et Outils Utilisés

Système d'exploitation	Windows 10+
Docker Engine	Version 24+
Docker Compose	Version 3.8
Serveur Web	Nginx (image alpine)
Base de données	PostgreSQL 13
Interface d'administration	pgAdmin 4
Éditeur de code	Visual Studio Code

3. Structure du Projet

La structure du répertoire du projet a été organisée de manière à séparer clairement le frontend du backend, conformément aux bonnes pratiques de développement conteneurisé :

```
project/
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   └── .dockerignore
├── backend/
│   └── db.env
└── docker-compose.yml
```

Cette organisation assure une séparation nette des responsabilités et facilite la maintenance ainsi que l'évolution du projet.

4. Réalisation des Étapes

Étape 1 : Création du Frontend

4.1.1 Fichier index.html

Le fichier HTML de la page d'accueil a été créé dans le dossier frontend/ :

```html
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Dockerized App</title>
</head>
<body>
 <h1>Bienvenue dans l'application Dockerisée</h1>
</body>
</html>
```

Étape 2 : Création du Dockerfile

4.2.1 Dockerfile du Frontend

Le Dockerfile utilise l'image officielle nginx:alpine pour sa légèreté et ses bonnes performances. Il copie le fichier HTML dans le répertoire de publication Nginx et expose le port 80 :

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4.2.2 Fichier .dockerignore

Le fichier .dockerignore permet d'exclure les fichiers inutiles du contexte de construction de l'image, allégeant ainsi le processus de build :

```
.log
node_modules/
.git/
```

Étape 3 : Configuration de la Base de Données

4.3.1 Fichier db.env

Les variables d'environnement pour PostgreSQL sont définies dans un fichier dédié backend/db.env afin de ne pas exposer les identifiants directement dans le docker-compose.yml :

```

POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp

```

Cette approche respecte les bonnes pratiques de sécurité en séparant la configuration sensible du code source.

Étape 4 : Fichier docker-compose.yml

Le fichier docker-compose.yml orchestre trois services interconnectés via un réseau dédié app-network :

```yaml
version: "3.8"
Run All Services
services:
  Run Service
  frontend:
    build:
      context: ./Front-end
      dockerfile: DockerFile
    ports:
      - "8081:80"
    networks:
      - app-network
  Run Service
  db:
    image: postgres:13
    environment:
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=${POSTGRES_DB}
    env_file:
      - ./backend/db.env
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network
  Run Service
  pgadmin:
    image: dpage/pgadmin4
    environment:
      - PGADMIN_DEFAULT_EMAIL=admin@admin.com
      - PGADMIN_DEFAULT_PASSWORD=admin
    ports:
      - "5050:80"
    depends_on:
      - db
    networks:
      - app-network
volumes:
  db-data:
networks:
```

Explication des choix de configuration :

- Le service frontend est construit à partir du Dockerfile local et est accessible sur le port 8080 de la machine hôte.
- Le service db utilise l'image PostgreSQL 13 officielle avec un volume nommé db-data pour assurer la persistance des données même après l'arrêt des conteneurs.
- Le service pgadmin dépend du service db (depends_on) pour garantir que la base de données est démarrée avant l'interface d'administration.
- Tous les services communiquent via le réseau virtuel app-network, ce qui les isole du reste du système.

Étape 5 : Tests et Validation

4.5.1 Démarrage des conteneurs

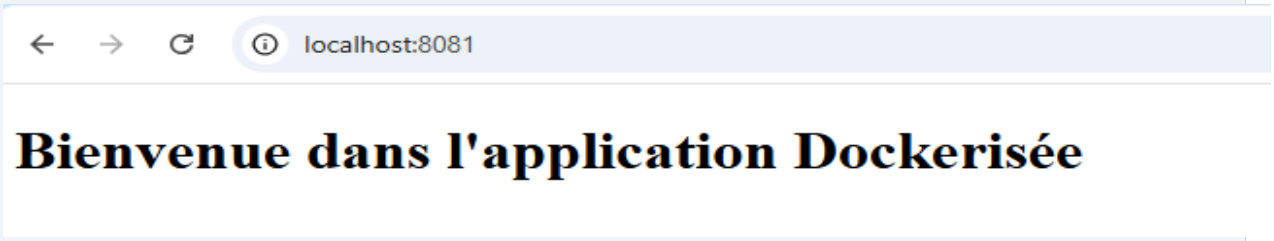
La commande suivante permet de construire les images et de démarrer tous les services :

```
PS C:\Users\user\OneDrive\Bureau\TPs_GL\TP_Docker> cd c:\Users\user\OneDrive\Bureau\TPs_GL\TP_Docker\Project
PS C:\Users\user\OneDrive\Bureau\TPs_GL\TP_Docker\Project> docker-compose up --build
time="2026-04-25T17:48:20+02:00" level=warning msg="The \"POSTGRES_USER\" variable is not set. Defaulting to a blank string."
time="2026-04-25T17:48:20+02:00" level=warning msg="The \"POSTGRES_PASSWORD\" variable is not set. Defaulting to a blank string."
time="2026-04-25T17:48:20+02:00" level=warning msg="The \"POSTGRES_DB\" variable is not set. Defaulting to a blank string."
time="2026-04-25T17:48:20+02:00" level=warning msg="C:\\Users\\user\\OneDrive\\Bureau\\TPs_GL\\TP_Docker\\Project\\docker-compose.yml: the attr
ibute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 605B 0.1s done
#1 DONE 0.1s
```

4.5.2 Vérification du Frontend

Après le démarrage, le frontend est accessible via le navigateur à l'adresse suivante :

```
http://localhost:8080
```



La page affiche correctement le message : « Bienvenue dans l'application Dockerisée ».

4.5.3 Connexion à pgAdmin

L'interface pgAdmin est accessible à l'adresse http://localhost:5050 avec les identifiants suivants :

Email	admin@admin.com
Mot de passe	admin

4.5.4 Ajout de la connexion PostgreSQL dans pgAdmin

Dans pgAdmin, une nouvelle connexion au serveur PostgreSQL a été configurée avec les paramètres suivants :

Host	db
Port	5432
Username	admin
Password	adminpassword
Database	myapp

Le hostname db correspond au nom du service défini dans docker-compose.yml, Docker résolvant automatiquement ce nom via le réseau app-network.

5. Persistance des Données

La persistance des données PostgreSQL est assurée par un volume nommé db-data déclaré dans docker-compose.yml. Ce volume survit aux redémarrages et suppressions des conteneurs, garantissant l'intégrité des données.

Pour vérifier la persistance :

```
# Arrêter et supprimer les conteneurs
docker-compose down

# Redémarrer les services
docker-compose up -d

# Les données sont toujours présentes dans le volume db-data
```

6. Analyse et Résultats

Frontend (Nginx)	Opérationnel - Port 8080
Base de données (PostgreSQL)	Opérationnel - Port 5432 (interne)
pgAdmin	Opérationnel - Port 5050
Volume de persistance	Fonctionnel (db-data)
Réseau inter-services	Fonctionnel (app-network)
Isolation des secrets	Effective (db.env)

Points Positifs

- Séparation claire des responsabilités (frontend / backend / administration).
- Utilisation de l'image nginx:alpine légère et optimisée.
- Isolation réseau efficace via app-network.
- Persistance des données garantie par le volume Docker.
- Gestion sécurisée des secrets via env_file.

Améliorations Possibles

- Ajouter un health check sur le service db pour s'assurer qu'il est pleinement opérationnel avant pgAdmin.
- Utiliser des secrets Docker ou un gestionnaire de secrets externe (Vault) pour davantage de sécurité.
- Mettre en place un reverse proxy (ex. Traefik) pour gérer les ports et le SSL.
- Ajouter un service backend applicatif (Node.js, Python Flask...) connecté à PostgreSQL.

7. Conclusion

Ce travail pratique a permis de mettre en œuvre une architecture multi-conteneurs complète et fonctionnelle avec Docker. La maîtrise du Dockerfile, du fichier .dockerignore, des volumes, des réseaux et de Docker Compose constitue une base solide pour le développement et le déploiement d'applications modernes.

La conteneurisation offre de nombreux avantages dans un contexte de transformation digitale industrielle : portabilité, reproductibilité, isolation et facilité de déploiement. Ces compétences s'inscrivent directement dans les métiers DevOps et Cloud que prépare la filière Transformation Digitale Industrielle.

Fouad BANAN | Transformation Digitale Industrielle | ENSA-BM